Introduction

Along these tutorials, we will see how **SCOPE** interacts with the different parts of the code to handle the execution of computational workflows.

These are the topics covered in each tutorial:

1. The **System** class and its sources: the **Specie**, **Cell** and **Atom** classes
2. The Computational workflow: **Branch**, **Workflow**, **Job**, and **Computation** classes
3. The **State** class
4. The **Data**, **Collection** and **VNM** classes
5. The **Input_data** class, and **scope input files**
6. Running **SCOPE** - Part 1: File Structure
7. Running **SCOPE** - Part 2: Execution
8. Running **SCOPE** - Part 3: Detailed Actions
9. The Azo add-on: Subclasses, Analysis and Creation of Azo Systems

Tutorial 5: Scope Input files

In this tutorial, we will have a look at an input file of SCOPE. The input is how we indicate Scope to interact with a **System** and perform a task. It is used both to (1) create the **Computations** that are submitted to a computational cluster, (2) to analyze the results of those **Computations**.

Scope Input Files are interpreted by Scope through the **Input_data** class. In the Data folder, these input files have the .scope extension. Importantly, they're not to be confused with the input files that Scope generates to submit Quantum Espresso (.inp) or Gaussian16 (.com) computations.

Below, we will create the scope input file for a task that we will execute in Tutorial 7

In [1]:
```
import os
import scope
```

In [2]:
```
## Path of Tutorial 7's Data folder. Normally, it should be "os.path.abspath
tutorial_folder = os.path.abspath('../')+'/Data/7-Tutorial_7/'
```

Loading [MathJax]/extensions/Safe.js

SCOPE Input Files

In [3]:
```
from scope.classes_input import *
```

In [4]:
```
## Input Files can be created from a file, or from a string.
## This is an example input for a Gaussian 16 computation, read as a string.
## At the end of this tutorial, you will find an example of a Quantum Espres

## First, notice that there are four sections, initiated with '&', and termi
## Sections can be empty as in the example below (see &options) or even abse
## Also, notice that the values are either introduced as strings 'X', lists
## Please respect the notation used for each type of variable:
   ## ''         for strings
   ## []         for lists
   ## True/False for booleans

## Finally, lines starting with '#' will be ignored. See environment sectior

test_input = """
&environment
   max_jobs         = 24
   max_procs        = 24
   requested_procs  = 1
   #queue            = 'my_preferred_one'
/

&options
/

&job_data
   branch       = 'tutorial'
   workflow     = 'all_sources'
   job          = 'scf'

   hierarchy    = 1
   requisites   = []
   constrains   = ['self']
   must_be_good = True

   istate       = 'initial'
   fstate       = 'initial'
/

&qc_data
   software     = 'g16'
   comp_type    = 'scf'
   functional   = 'pbe'
   basis        = 'sto-3g'
/"""
```

In [5]:
```
# IMPORTANT. Notice that, when running the 'scope_run_input' script. The inp
# save this text to an actual file, so we can call it later
from scope.read_write import save_text
```

Loading [MathJax]/extensions/Safe.js

```
file_name = ''.join([tutorial_folder,"task1.scope"])
save_text(test_input, file_name)
```

In [8]:
```
## In any case, the input is interpreted and stored as INPUT_DATA class obje
local_env, options, job_data, qc_data = set_input_data(content=test_input, i
```

```
INPUT_DATA.READ: Section '&options' is empty. Only defaults will be taken
```

Section 1: Local Environment

In [9]:
```
# During the creation of the Environment-class objects (the 'global' enviror
# Typically, these are the queues that the user has access to during the pro

# Some other choices are more Job-dependent, like the requested number of pr
# To facilitate interaction, these options can be specified here in the inpu
# without having to load and modify the 'global' environment

## These are the user choices in the input file:
local_env
```

Out[9]:
```
Formatted input interpretation: ( self -> Instance of class Input() )
----------------------------------------------------
self.Key                  | Data Type           | Value
----------------------------------------------------
self.max_jobs             | <class 'int'>       | 24
self.max_procs            | <class 'int'>       | 24
self.requested_procs      | <class 'int'>       | 1
self.method               | <class 'str'>       | weighted
```

In [10]:
```
# max_jobs:
    # Maximum number of SLURM jobs running simultaneously by the user.
# max_procs:
    # Maximum number of processors that can be used simultaneously by the us
# requested_procs:
    # Ideal number of processors to be requested in this Job's computations
```

In [11]:
```
## In the input file, we commented the variable 'queue':

## queue:
    # Optional, can be used to limit the actual queues that are accessible t
    # If not specified, the 'best' queue will be selected among the ones mac
    # Normally, the 'best' queue is determined based on the number of availa
```

Section 2: Options

In [ ]:
```
## These are other options related to the submission:
options = fill_options_data(options)
print(options)

## This section was empty in the "test_input", but now defaults were added:

# want_submit:
    , the user can turn off submission (want_submit = False) of jobs.
    # If so, input files will be created, but computations won't be submitte
```

Loading [MathJax]/extensions/Safe.js

```
    # This is perfect for testing the input files creation

# ignore_submitted:
    # Normally when submitting a computation, SCOPE verifies that no computa
    # If activated (ignore_submitted = True), this check won't be done, and
    # This might be OK in some cases, when those job, even if they have the

# overwrite_inputs and overwrite_outputs:
    # When true, the code can overwrite files that already exist.
```

```
Formatted input interpretation: ( self -> Instance of class Input() )
----------------------------------------------------
self.Key                  | Data Type           | Value
----------------------------------------------------
self.want_submit          | <class 'bool'>      | True
self.ignore_submitted     | <class 'bool'>      | False
self.overwrite_inputs     | <class 'bool'>      | True
self.overwrite_outputs    | <class 'bool'>      | True
```

Section 3: Job Data Section

In [14]:
```
## This section includes the information about the JOB, and can be divided i
# - How the JOB fits in the overall workflow
# - JOB options
# - How is the JOB related to the States

## Basically, a Job associated with this information will be created (if it
job_data = fill_job_data(job_data)
print(job_data)
```

```
Formatted input interpretation: ( self -> Instance of class Input() )
----------------------------------------------------
self.Key                  | Data Type           | Value
----------------------------------------------------
self.branch               | <class 'str'>       | tutorial
self.workflow             | <class 'str'>       | all_sources
self.job                  | <class 'str'>       | scf
self.hierarchy            | <class 'int'>       | 1
self.requisites           | <class 'list'>      | []
self.constrains           | <class 'list'>      | ['self']
self.must_be_good         | <class 'bool'>      | True
self.istate               | <class 'str'>       | initial
self.fstate               | <class 'str'>       | initial
self.job_setup            | <class 'str'>       | regular
self.job_type             | <class 'str'>       | scf
```

In [ ]:
```
## Here's the meaning of each variable:

# --- How the JOB fits in the overall workflow ---
# branch:       to locate the job inside the system, the BRANCH must be spec
# workflow:     to locate the job inside the system, the WORKFLOW must be sp
# job:          Name given to this job. Use to search it, and to define the
# hierarchy:    expected order of the job inside the workflow
# requisites:   list of job_names that should have finished before this one
```

Loading [MathJax]/extensions/Safe.js

```
# constrains:   list of job_names that should have NOT finished before this

# --- How is the JOB related to the States ---
# istate:       state from which the information must be extracted
# fstate:       state where the information will be stored

# --- JOB Options ---
# must_be_good: whether the job must finish correctly to be considered compl
    # if must_be_good = True, it will try hard to do the task. For instance,
    # if must_be_good = False, it will perform just one attempt
# comp_type:    Type of Computation. Implemented are ["scf", "opt", "freq",
```

Section 4: QC data

In [ ]:
```
## Finally, we have the information of the actual Quantum Chemistry (QC) com
qc_data = fill_qc_data(qc_data)
print(qc_data)

## This is the information that will be passed to the QC software to create
## The variables here might be software-dependent. Those in the example abov
## Below, you will find an example of a Quantum Espresso input file.

## In principle, you can update the information here (eg. you can change the
## If you do so, and everything works well, the code will compare the old ar
## ...it will resubmit even if the same job under the old qc_data was flagge
## In other words, it should be safe to update the qc_data, but we haven't t
```

```
Formatted input interpretation: ( self -> Instance of class Input() )
-----------------------------------------------------
self.Key                  | Data Type           | Value
-----------------------------------------------------
self.software             | <class 'str'>       | g16
self.comp_type            | <class 'str'>       | scf
self.functional           | <class 'str'>       | pbe
self.basis                | <class 'str'>       | sto-3g
self.loose_opt            | <class 'bool'>      | False
self.tight_opt            | <class 'bool'>      | False
self.is_grimme            | <class 'bool'>      | False
self.grimme_type          | <class 'str'>       | d2
self.ultrafine_grid       | <class 'bool'>      | False
```

Quantum Espresso input file example

In [ ]:
```
## See below for an example of the qc_data for Quantum Espresso:

#&qc_data
#   software     = 'qe'              ## software to be used. Currently, onl
#   version      = '7.0'             ## version of the software. For Quantu
#   comp_type    = 'vc-relax'        ## Type of calculation. Implemented ar
#                                    ##  - ["scf", "opt", "freq", "ts", "tc
#                                    ##  - ["scf", "opt"="relax", "vc-relax
#   functional   = 'pbe'             ## Only GGA (PBE) available for now.
#   is_hubbard   = True              ## Activation of Hubbard correction. I
```

Loading [MathJax]/extensions/Safe.js

```
#   is_grimme    = True              ## Activation of Grimme's dispersion c
#   grimme_type  = 'd3bj'            ## Options are "d2", "d3", "d3bj", "d3
#   uterm        = 2.35              ## U term for Hubbard correction. Only
#   mix_beta     = 0.7               ## Mixing beta for the SCF. Might be a
#   elec_conv    = 1e-5              ## Electronic convergence threshold in
#   pseudo       = 'vanderbilt'      ## Pseudo-Potential library to be used
#/
```

In [15]:
```
## Notice that we have implemented different Pseudo-Potential Libraries avai
## These were extracted from the internet, mostly from GBRV: https://www.phy
## and QE's official website: https://www.quantum-espresso.org/pseudopotenti
```

Note on the limited options for QC computations (basis sets, functionals, keywords, in qc_data)

Currently Only Quantum Espresso and Gaussian16 packages are implemented together with a very limited set of functionals, basis sets, etc...
So... we are aware that the code accepts a very limited set of qc_data options and software.

Initially, our priority was to implement the ones that we needed for our ongoing work. Then, our priority was to organize and publish the code, which is what we are doing now. Similarly, we focused on the active versions of G16 and QE by the time of the implementation. Other versions will be implemented in future updates.

In any case, the implementation of functionals, basis sets, and other G16 or QE options is relatively simple, please do not hesitate to write me at sergi.vela@iqac.csic.es.
In the meantime, please accept our apologies.

Loading [MathJax]/extensions/Safe.js